

Recommendation from other CHAT GPT6 Model

Charging Data Plane, Performance, and HA Architecture

Objective

Design and incrementally implement a low-latency, high-availability charging data plane for this 5G Core and CHF.

During charging decisions, CHF must have the complete information it needs in memory: subscriber/account and bucket mapping, applicable active products and prices, rating rules, currency and rounding rules, session reservations, usage counters, and the versions of the catalog and policy used. Do not make persistent-store reads or per-request calls to product-catalog or balance-management to calculate a charge.

This requirement does not remove the need for durable writes. A new reservation or settlement must not be acknowledged until it has met the explicitly chosen durability guarantee.

Keep the existing open-source stack where it remains suitable:

- Valkey for hot/shared operational state.
- PostgreSQL for authoritative monetary balances, reservations, and ledger.
- Doris for CDR analytics and reporting.
- Kafka only where asynchronous event distribution is needed.

Do not add a database product without measuring a concrete benefit and recording the infrastructure and operational cost.

Current Findings To Verify Against HEAD

Start by checking the repository's current branch and inspecting the actual implementations, configuration, schemas, tests, Compose services, and ADRs. Do not rely on old status tables without verifying them against source.

The current public snapshot has these relevant issues:

- CHF rating fetches the ProductOffering collection and then fetches candidate prices individually over SBI.

- CHF resolves subscriber buckets and reserves/finalizes balance through separate SBI calls.
- Balance reservation is atomic within PostgreSQL, but release settlement currently uses separate unreserve and debit operations, leaving a failure window.
- Balance amounts pass through `double` in C++ despite PostgreSQL storing `NUMERIC`; review and replace binary floating-point money arithmetic with exact decimal or integer minor-unit arithmetic.
- CHF's CDR event producer enqueues asynchronously; `acks=all` is configured, but `publish()` does not wait for the broker's delivery report before returning.
- UDR constructs dozens of store objects with individual PostgreSQL connections; the current count is approximately 88 connections per process. UDR should use a bounded shared connection pool.
- The shared PostgreSQL pool waits indefinitely when exhausted; add acquisition deadlines, back-pressure, and metrics.
- PostgreSQL is not configured as a replicated/failover deployment in Compose. The new Valkey Cluster is additive, has AOF disabled, and is not used by the default NF configuration.
- Domain DDL is primarily applied on first initialization of an empty database volume. A production schema migration and rollback/forward-recovery process is needed.
- `docs/DATA\_MODEL.md` is marked DRAFT and its proposed Valkey balance role does not match the current PostgreSQL-only balance implementation. Reconcile the model, code, and stated guarantees.

Treat these as findings to verify, not assumptions to blindly implement.

Required Architecture Decisions

Before changing runtime code, write or update an ADR that compares at least these designs:

1. PostgreSQL authoritative balance with in-memory CHF rating snapshots.
2. Valkey as a replicated hot-state projection with PostgreSQL authoritative ledger.
3. Valkey as an authoritative reservation/balance store backed by a durable journal or equivalent recovery mechanism.

For each, analyze:

- Per-request read/write path and expected latency.
- Atomicity for concurrent requests against one balance or shared account.
- Behavior on CHF restart, Valkey failover, PostgreSQL failover, network partition, and replay.

- Duplicate requests and idempotency-key retention.
- Write durability and the point at which CHF may acknowledge a reservation.
- Cross-region behavior and achievable RPO/RTO.
- Infrastructure footprint and operational complexity.

Do not describe asynchronous Valkey-to-PostgreSQL replication as lossless or strongly consistent. Do not update two independent stores without an explicit recovery protocol.

Target Charging Flow

Implement the following shape, adapting it to the selected ADR:

1. Build a versioned in-memory charging snapshot from authoritative PostgreSQL data. Include the complete read set required for rating; do not perform catalog or balance reads during a charging interrogation.
2. Refresh snapshots on catalog/policy changes using versioned events or a bounded refresh strategy. Provide startup warm-up, version checks, cache invalidation, and full rebuild after gaps.
3. At session creation or reauthorization, compute rating from the in-memory snapshot and reserve quota through one atomic, idempotent operation.
4. Keep per-session grants, usage counters, reservation totals, invocation sequences, catalog version, and policy version in shared hot state.
5. Let the UPF meter packets locally. Report usage at defined thresholds and reauthorize before granted quota is exhausted. Do not persist a database row per packet.
6. Settle a reservation through one atomic balance-service operation that updates remaining/reserved value and appends the ledger event in the same transaction. Make retries with the same idempotency key return the original result.
7. Persist CDRs and rating decisions through a durable write path. Prefer a transactional outbox: commit the charge/ledger/outbox record, then deliver to Kafka and Doris asynchronously with replay and deduplication.
8. If the authoritative reservation cannot be committed, do not grant unlimited service. Define configurable fail-open/fail-closed policy explicitly; any continued service must be bounded by already granted quota.

The feature requirement is that rating does not depend on database reads. A durable write may still be required for reservation, settlement, and audit.

Data Model and Scaling Requirements

- Keep the PostgreSQL balance ledger authoritative unless the ADR proves another design has equivalent atomicity and recovery.
- Use fixed-precision money types end-to-end; define currency scale, rounding mode, overflow behavior, and serialization rules.
- Add a stable reservation/idempotency record tying SUPI/account, bucket, chargingDataRef, invocation sequence, amount, granted units, consumed units, tariff version, and final outcome.
- Partition hot mutable state by account or bucket owner. All mutations for a shared/enterprise balance must have one serialized owner or an equivalent atomic compare-and-update mechanism.
- Use version/fencing tokens to prevent a former owner from writing after failover.
- Treat in-memory rating snapshots as rebuildable projections. Record source version and expose snapshot age/version metrics.
- Keep large immutable CDR history in Doris/object storage as appropriate; don't use Doris as the synchronous balance authority.
- Add indexes from measured query plans. Avoid blanket GIN indexes on every JSONB document.
- Convert UDR to a bounded shared PostgreSQL pool; remove the per-store connection explosion.
- Bound pool waits and queues. Return explicit overload/failure behavior rather than hanging worker threads indefinitely.
- Create a versioned migration framework for PostgreSQL and Doris schemas. Test upgrades from a populated prior schema, not only fresh-volume initialization.

HA and Low-Infrastructure Profiles

Provide separate deployment profiles:

- **Minimal/dev:** smallest useful PostgreSQL and Valkey footprint; Kafka, Doris analytics, GUI, and other optional components are disabled unless needed. State clearly that this profile is not HA.
- **Production/HA:** replicated PostgreSQL with tested failover and PITR, durable Valkey deployment with the selected persistence policy, replicated Kafka only for enabled event flows, and Doris topology/backup appropriate to its analytics role.
- The default deployment must not start unused six-node Valkey or three-broker Kafka infrastructure merely because those services are defined.
- Add health/readiness semantics for every datastore. Distinguish “process alive” from “ready to accept charging.”
- Test graceful shutdown, startup ordering, failover, restore, and rolling upgrades.

Performance and Reliability Evidence

Establish a repeatable baseline before and after each performance change. Measure:

- Nchf Create, Update, Release, reauthorization, balance reservation, and settlement separately.
- p50/p95/p99/p99.9 latency, throughput, errors, CPU, RSS, connection counts, and pool wait time.
- Small and production-shaped catalog sizes, normal and large subscriber counts, single-SUPI contention, shared-account contention, and multiple CHF replicas.
- Cold-cache warm-up, cache refresh/version changes, replay, and duplicate requests.
- Failure cases: kill CHF after reservation, kill Valkey leader, restart PostgreSQL primary, partition the network, stop Kafka/Doris, and replay the same request.
- Assert no overdraft, no duplicate debit, no unbounded free service, no lost acknowledged settlement, and deterministic recovery.
- Publish the test hardware, configuration, workload, result spread, and unverified conditions.

Do not claim carrier-grade, lossless, or vendor-superior behavior without those results. Define explicit RPO/RTO and load/soak targets before calling HA complete.

Incremental Execution

Do not implement the entire architecture in one change. First:

1. Inspect current HEAD and trace the live charging flow from SMF through CHF, catalog, balance, Valkey, PostgreSQL, Kafka, and Doris.
2. Write the ADR comparing the storage/consistency options and provide the proposed request sequence and failure semantics.
3. Propose a measurable baseline and focused tests.

Then implement one subsystem per increment, with tests and an ADR/doc update for each. The first implementation candidate should be CHF's in-memory versioned rating snapshot and indexed lookup, because it removes repeated catalog reads without changing monetary source-of-truth semantics. Next, implement atomic, idempotent reservation settlement and its recovery tests. Only then expand Valkey HA and outbox/Doris delivery.

Do not change balance consistency semantics, fail-open behavior, schema source-of-truth rules, or deployment scope without recording the decision and its tradeoffs.

After CHF implementation, follow the same for all NF which required persistence and in memory requirements for performance and audit (Example UDF,UDR etc) . The same should be reflected on GUI design.

DO NOT USE/RUN MULTI-AGENT now as Datastore design should finalized before any GUI or other changes in PARALLEL

Please review and give feedback and update ALL documentations- This is a must for tracking and implementations